ELDORIA:
TREASURES OF THE FALLEN KINGDOM

Memoria del proyecto – Versión 1.1

Autor: Sergio Arévalo Hidalgo

Curso: 1.º de Desarrollo de Aplicaciones Web (DAW)

Lenguaje utilizado: Java

Repositorio: https://github.com/Checho2525/Eldoria

ÍNDICE

1. INTRODUCCIÓN
2. OBJETIVOS DEL PROYECTO
3. ¿EN QUÉ CONSISTE EL JUEGO?
4. REGLAS Y FUNCIONAMIENTO
5. FUNCIONALIDADES DEL JUEGO
6. CÓMO ESTÁ HECHO EL PROGRAMA
7. PRUEBAS DEL PROGRAMA
8. PROBLEMAS ENCONTRADOS Y SOLUCIONES
9. IDEAS PARA FUTURAS VERSIONES
10. CONCLUSIÓN

1. INTRODUCCIÓN

Para este proyecto he desarrollado Eldoria: Treasures of the Fallen Kingdom, un videojuego de rol (RPG) hecho en Java que funciona desde la consola.

La idea del juego es controlar a un personaje que avanza por diferentes salas y se enfrenta a enemigos. Cuanto más avanza, más difíciles se vuelven los combates.

Durante las partidas se puede atacar, defenderse o intentar huir. Al derrotar a un enemigo se consigue oro y experiencia, y se recupera vida. Cuando se alcanza la experiencia necesaria, el personaje sube de nivel y mejora sus estadísticas.

El objetivo del videojuego es llegar lo más lejos posible. En esta versión no hay una última sala ni una victoria final: la partida termina cuando el personaje se queda sin vida o cuando el jugador decide salir.

2. OBJETIVOS DEL PROYECTO

2.1. Objetivo del videojuego

El objetivo dentro del juego es superar el mayor número posible de salas, derrotando enemigos, consiguiendo oro y experiencia y mejorando las estadísticas del personaje.

2.2. Objetivos de programación

El objetivo de programación es aplicar los contenidos trabajados durante la primera evaluación y conseguir que el juego funcione de forma ordenada y controlada.

- Crear un menú principal que permita comenzar la aventura, consultar las estadísticas y salir.
- Programar combates por turnos contra distintos enemigos.
- Permitir que el jugador ataque, se defienda o huya.
- Actualizar la vida, el oro, la experiencia, el nivel y la sala según las acciones realizadas.
- Utilizar arrays para almacenar y procesar estadísticas.
- Validar las opciones numéricas incorrectas y controlar la derrota.

3. ¿EN QUÉ CONSISTE EL JUEGO?

Al abrir Eldoria aparece un menú principal con tres opciones: batallar en la mazmorra, consultar las estadísticas y salir.

Al comenzar un combate, el programa selecciona aleatoriamente uno de los tres enemigos disponibles: Slime, Goblin o Esqueleto. Cada enemigo tiene sus propias estadísticas y estas aumentan según la sala.

3.1. Estadísticas iniciales del personaje

Estadística	Valor inicial
Vida	50
Daño	10
Oro	0
Nivel	1
Experiencia	0

La vida indica cuánto puede aguantar el personaje, el daño determina cuánto resta al enemigo y el oro representa las recompensas conseguidas. El nivel y la experiencia indican la progresión del personaje.

3.2. Enemigos

Enemigo	Vida inicial	Daño inicial	Oro
Slime	20	7	5
Goblin	20	6	6
Esqueleto	25	7	7

El enemigo de cada combate se elige al azar, por lo que no siempre aparece el mismo.

4. REGLAS Y FUNCIONAMIENTO

4.1. Los combates

Los combates funcionan por turnos. Primero el jugador elige una acción y después, si el enemigo sigue vivo, este realiza su ataque.

- Atacar: el personaje resta vida al enemigo según su daño actual.
- Defenderse: reduce a la mitad el daño del siguiente ataque enemigo. La división se realiza con números enteros; por ejemplo, 7 puntos de daño pasan a ser 3.
- Huir: abandona el combate y vuelve al menú principal. No se reciben las recompensas de ese enemigo.

Al huir se conservan la vida, el oro, la experiencia y la sala que tenía el personaje. La aventura no se reinicia. Si se introduce una acción numérica incorrecta, el programa vuelve a pedirla y el enemigo no ataca por ese error.

4.2. Recompensas y recuperación de vida

Al derrotar a un enemigo se consigue el oro correspondiente y 10 puntos de experiencia. Además, el personaje recupera 10 puntos de vida. La vida nunca supera el máximo permitido: si la recuperación sobrepasaría ese límite, se ajusta al máximo.

4.3. Subir de nivel

Para subir de nivel hacen falta 20 puntos de experiencia al principio. Al alcanzar el requisito, se aumenta un nivel y se descuentan los puntos necesarios. Para el siguiente nivel hacen falta 10 puntos más que para el anterior.

- La vida máxima aumenta en 10 puntos.
- El daño aumenta en 4 puntos.
- El personaje recupera 10 puntos de vida, sin superar la nueva vida máxima.

Ejemplo: al pasar del nivel 1 al nivel 2, la vida máxima pasa de 50 a 60 y el daño pasa de 10 a 14. Si el personaje tenía 40 puntos de vida justo antes de subir, la recuperación de nivel lo deja con 50 puntos, siempre que no exista otro límite que lo impida.

4.4. Dificultad de las salas

Por cada sala posterior a la primera, las estadísticas del enemigo aumentan respecto a sus valores iniciales: 5 puntos de vida, 2 puntos de daño y 2 monedas de oro por cada sala avanzada.

La fórmula aplicada es: valor del enemigo = valor inicial + (sala − 1) × aumento por sala. Por ejemplo, un Slime en la sala 2 tiene 25 puntos de vida, hace 9 puntos de daño y entrega 7 monedas de oro.

4.5. Cómo termina una partida

La partida termina cuando la vida del personaje llega a cero. El programa muestra el mensaje de derrota y finaliza. No existe una victoria final en esta versión: el objetivo es avanzar por tantas salas como sea posible.

Si el jugador huye, vuelve al menú principal y puede continuar la misma aventura más adelante, conservando el estado del personaje y la sala.

5. FUNCIONALIDADES DEL JUEGO

Funcionalidad	Descripción	Funcionamiento
Menú principal	Permite acceder a las opciones principales.	Un bucle mantiene el menú activo hasta salir o perder.
Combate por turnos	Permite atacar, defenderse o huir.	El jugador elige una acción y el enemigo responde si sigue vivo.
Enemigos aleatorios	Hay tres tipos de enemigos.	Math.random() selecciona uno de los tres.
Recompensas	Se obtiene oro y experiencia al ganar.	Los valores del personaje se actualizan tras la victoria.
Recuperación de vida	Se recupera vida al vencer y al subir de nivel.	La vida se limita al máximo permitido.
Subida de nivel	Mejora las estadísticas del personaje.	Se comprueba la experiencia necesaria y se aplican las mejoras.
Dificultad progresiva	Los enemigos aumentan sus estadísticas por sala.	Se aplica un incremento según el número de sala.
Consulta de estadísticas	Muestra la información del personaje y la partida.	Se recorren los datos del array del personaje y se muestran.
Colores de consola	Distinguen visualmente los mensajes.	Se utilizan códigos ANSI almacenados en constantes.

6. CÓMO ESTÁ HECHO EL PROGRAMA

El programa está desarrollado en Java y el código se encuentra dentro de una clase llamada Eldoria. Para esta evaluación se han utilizado tipos de datos, variables, constantes, arrays, condicionales, bucles y entrada de datos.

6.1. Variables

Las variables individuales guardan el estado general de la partida. El array personaje guarda tres estadísticas relacionadas del jugador.

Variable	Tipo	Uso
respuesta	int	Opción elegida en el menú.
accion	int	Acción elegida durante el combate.
nivel	int	Nivel actual del personaje.

experiencia	int	Experiencia acumulada.
experienciaNecesaria	int	Experiencia necesaria para subir de nivel.
vidaMaxima	int	Máximo de vida permitido.
sala	int	Sala actual de la aventura.
vivo	boolean	Indica si la partida continúa.
enemigoAleatorio	int	Índice del enemigo seleccionado.
vidaEnemigo / dañoEnemigo / oroEnemigo	int	Estadísticas actuales del enemigo.
defenderse	boolean	Indica si se ha elegido defenderse.
personaje	int[]	Array con vida, daño y oro del personaje.

6.2. Constantes

Las constantes se declaran con final porque sus valores no deben cambiar durante la ejecución. Se usan nombres en mayúsculas para distinguirlas de las variables. Además de los códigos de color, se utilizan constantes para los incrementos y recompensas del juego.

Constante	Función
CURACION_VICTORIA	Puntos de vida recuperados al ganar.
EXPERIENCIA_VICTORIA	Experiencia obtenida al ganar.
MEJORA_DANO_NIVEL	Aumento de daño al subir de nivel.
MEJORA_VIDA_MAXIMA_NIVEL	Aumento de vida máxima al subir de nivel.
AUMENTO_VIDA_ENEMIGO_SALA	Aumento de vida del enemigo por sala.
AUMENTO_DAÑO_ENEMIGO_SALA	Aumento de daño del enemigo por sala.
AUMENTO_ORO_ENEMIGO_SALA	Aumento de oro del enemigo por sala.

6.3. Arrays

El array personaje almacena tres valores: la posición 0 contiene la vida, la posición 1 el daño y la posición 2 el oro. También existen arrays para las estadísticas iniciales del Slime, el Goblin y el Esqueleto.

```
int[] personaje = {50, 10, 0};
```

Para mostrar las estadísticas se utiliza un array de nombres y un bucle for que recorre las posiciones del array personaje. La propiedad length indica cuántas posiciones tiene, por lo que el bucle se adapta al tamaño del array.

```
String[] nombres = {"VIDA", "DAÑO", "ORO"};
for (int i = 0; i < personaje.length; i++) {
    System.out.printf("* %-13s = %-10d *%n", nombres[i], personaje[i]);
}
```

En cada repetición, i indica la posición que se está consultando. Así se muestran los tres datos sin escribir una instrucción distinta para cada posición.

6.4. Condicionales

Se utilizan if, else if y else para comprobar la opción elegida, seleccionar el enemigo, calcular el daño y comprobar si el jugador o el enemigo han sido derrotados. También controlan la defensa, la subida de nivel y las opciones incorrectas.

6.5. Bucles

El bucle while principal mantiene abierto el menú hasta que el jugador sale o pierde. Otro while mantiene el combate activo mientras el personaje y el enemigo tengan vida. El bucle for recorre el array de estadísticas para mostrarlas.

break permite salir del combate cuando el jugador huye. continue hace que se solicite otra acción cuando la opción introducida no es válida, sin ejecutar el ataque del enemigo.

6.6. Entrada de datos

Se utiliza Scanner para leer los números introducidos por teclado mediante nextInt(). Las opciones numéricas se validan dentro de los valores permitidos. La entrada de letras todavía no está controlada y queda como mejora futura.

6.7. Números aleatorios

Math.random() genera un valor decimal mayor o igual que 0 y menor que 1. Al multiplicarlo por 3 y convertirlo a int, se obtiene uno de estos índices: 0, 1 o 2. Cada índice corresponde a Slime, Goblin o Esqueleto.

7. PRUEBAS DEL PROGRAMA

Se han definido pruebas para comprobar el menú, las acciones del combate y la progresión. Los resultados deben coincidir con el comportamiento real del programa. En los casos que dependen del enemigo aleatorio, el resultado concreto puede variar.

N.º	Datos / acción introducida	Resultado esperado	Resultado obtenido / comprobación
1	Elegir opción 1 en el menú.	Aparece un enemigo y comienza el combate.	El combate comienza y se muestra uno de los tres enemigos.
2	Elegir opción 2 en el menú.	Se muestran las estadísticas.	Se muestran vida, daño, oro, nivel, experiencia y sala.
3	Elegir opción 3 en el menú.	El programa termina de forma controlada.	El programa sale del menú y finaliza.
4	Introducir un número fuera de las opciones del menú.	Se muestra un aviso y se vuelve a pedir la opción.	La opción se rechaza y el menú vuelve a solicitar una entrada.
5	Elegir atacar.	La vida del enemigo disminuye según el daño del jugador.	El enemigo pierde los puntos de daño del personaje.
6	Elegir defenderse ante un ataque de 7 puntos.	El daño recibido se reduce a 3 por división entera.	Se reciben 3 puntos de daño.
7	Elegir huir.	Se abandona el combate sin recompensa y se vuelve al menú conservando el estado.	Se vuelve al menú y se mantiene el estado de la aventura.
8	Introducir una acción de combate no válida.	Se vuelve a pedir una acción y el enemigo no ataca por esa entrada.	Se solicita otra acción sin recibir daño por la opción incorrecta.
9	Derrotar a un enemigo.	Se añade el oro y 10 de experiencia; se recuperan 10 puntos de vida sin superar el máximo.	Se actualizan las recompensas y la vida respetando el máximo.

10	Alcanzar 20 puntos de experiencia en nivel 1.	Se sube al nivel 2; el daño aumenta 4 y la vida máxima 10.	El nivel y las estadísticas aumentan según las mejoras indicadas.
11	Combatir en sala 2 contra un Slime.	El Slime tiene 25 de vida, 9 de daño y recompensa de 7 de oro.	Las estadísticas se calculan aplicando el aumento de una sala.
12	Llegar a 0 puntos de vida.	Se muestra la derrota y termina el programa.	Se muestra el mensaje de derrota y la partida finaliza.

8. PROBLEMAS ENCONTRADOS Y SOLUCIONES

Durante el desarrollo aparecieron problemas relacionados con el control del menú, los turnos y la progresión. Para resolverlos se ajustaron las condiciones y los bucles del programa.

8.1. Mantener el menú activo

El menú debía aparecer después de cada combate o consulta. Se solucionó con un bucle while que se repite mientras la partida siga activa y el jugador no elija salir.

8.2. Terminar la partida al morir

Era necesario detener el juego cuando el personaje se quedase sin vida. Se controla la vida del personaje y se actualiza la variable booleana vivo para que el bucle principal termine.

8.3. Hacer funcionar la defensa

La defensa debía reducir el daño recibido sin mantenerse activa para siempre. Se aplica la reducción al ataque correspondiente y se reinicia el estado de defensa en el siguiente turno.

8.4. Controlar las opciones incorrectas

El enemigo no debía atacar cuando el jugador introducía una acción numérica no válida. Se utiliza continue para volver al inicio del ciclo y solicitar otra acción.

8.5. Sistema de experiencia

Había que comprobar cuándo se alcanzaba la experiencia necesaria y aplicar las mejoras correctamente. Se compara la experiencia acumulada con experienciaNecesaria y se actualizan el nivel, el requisito siguiente y las estadísticas.

9. IDEAS PARA FUTURAS VERSIONES

9.1. Segunda evaluación

- Organizar el programa mediante clases y objetos.
- Separar las responsabilidades del personaje y los enemigos en distintas clases.
- Estudiar la incorporación de herencia, polimorfismo y excepciones cuando se hayan trabajado en clase.

9.2. Tercera evaluación

- Valorar el uso de colecciones para gestionar más elementos.

- Guardar información en ficheros o en una base de datos.
- Añadir persistencia para poder guardar y recuperar partidas.

Estas mejoras quedan planteadas para las siguientes evaluaciones y no forman parte de la versión actual.

10. CONCLUSIÓN

Con este proyecto he podido poner en práctica los conceptos de Java trabajados durante la primera evaluación: variables, constantes, arrays, condicionales, bucles y entrada de datos.

Una de las partes que más trabajo me ha llevado ha sido el sistema de combate, porque había que controlar los turnos, los ataques, la defensa, las recompensas y la derrota. También he tenido que revisar el funcionamiento de los arrays y comprobar que se recorren correctamente.

Estoy satisfecho con el resultado porque he conseguido crear un juego desde cero y dejar una base que se puede ampliar en las siguientes evaluaciones. Como mejora pendiente, todavía se podría controlar la entrada de letras y ampliar las pruebas con más ejecuciones documentadas.